


Designed by Remus Teodorescu 2026

LECTURE 11.2

# Obstacle Avoidance

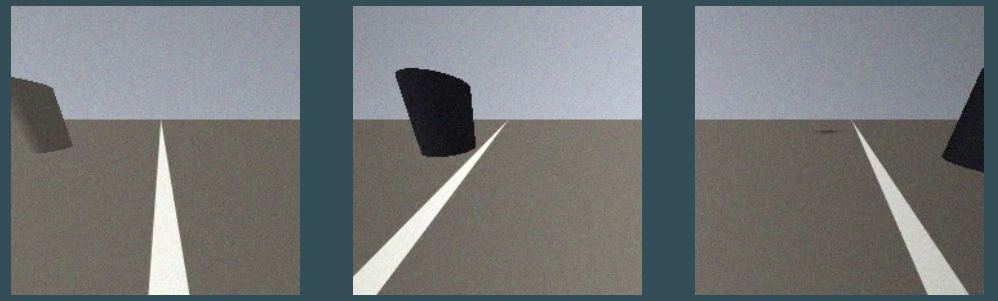
**Remus Teodorescu**

*With support from Research Assistant Noman Khan - nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

IDENTIFY · DECIDE · IN TIME



*free* *blocked* *a shadow: free*

*Something stands in the path. The lecture answers the two questions every autonomous vehicle must answer, first in words and then in mathematics: how do I identify the obstacle - a network that tells free from blocked, and what its errors cost - and how do I decide the action - stop, steer around or follow a field - in time to stop.*

# Learning Objectives

## SECTIONS, IN ORDER

1. **Identifying obstacles** — what blocks the path, and how a network tells

2. **Deciding the action** — stop, steer around, or follow a field

3. **Reacting in time** — the stopping distance, and a faster loop

*Each section answers its question twice: first in words, then in mathematics. The example is Ex_11's drawn frames and its A-to-B course, where the truth is known; the real JetRacer drives the same task in a lab demonstration.*

## READ, AND WHAT EACH ANSWERS

| | |
|---|---|
| **UDL ch. 5** | how do I identify an obstacle? |
| **UDL ch. 19** | how do I decide the action? |
| **PyTorch** | how fast must the loop be? |

# Identifying an Obstacle

**Obstacle** — anything that stands in the path ahead, near enough that the car must act on it. Its colour and shape do not decide it; where it stands does. A shadow on the floor is dark but solid nothing, so it is not an obstacle.

## THREE QUESTIONS ABOUT EVERY FRAME

| | |
|---|---|
| **is it in the lane?** | beside the path is not in the way |
| **is it near?** | far away can wait for the next frame |
| **is it solid?** | a shadow or a reflection is not |

**TALK · 2 MIN** How would you decide, in your own words, what counts as an obstacle for a car? Then, in the three ways on the right, which one tells the car where to steer?

## THREE WAYS TO SEE IT




| | the output | enough to |
|---|---|---|
| classify | free or blocked | stop or go |
| detect | a box: where, how big | steer around |
| segment | every free pixel | plan a path |

*Ex_11 classifies, as NVIDIA's JetBot does.*

# Free or Blocked

**Binary classification** — a network with two outputs, one for free and one for blocked, turned into a probability by the softmax. It is trained with cross-entropy on frames labelled by where the obstacle stands, so the network learns what makes a frame blocked.

## THE PROBABILITY, AND THE LOSS THAT TRAINS IT

$$p_{\mathrm{b}} = \frac{e^{z_{\mathrm{b}}}}{e^{z_{\mathrm{f}}} + e^{z_{\mathrm{b}}}}$$

$z_{\mathrm{f}}, z_{\mathrm{b}}$ the network's two outputs, $p_{\mathrm{b}}$ the probability of blocked

$$\mathcal{L} = -\frac{1}{N}\sum_{k=1}^{N}\left[y_k \ln p_{\mathrm{b},k} + (1-y_k)\ln(1-p_{\mathrm{b},k})\right]$$

$y_k$ 1 when frame $k$ is blocked, 0 when free, $N$ the frames in a batch

### THE HEAD AND THE LOSS

```
head = torch.nn.Linear(32, 2)                    # two outputs: free, blocked
p_b = torch.softmax(net(x), dim=1)[:, 1]         # the probability of blocked
loss = torch.nn.functional.cross_entropy(net(x), y)   # the loss above
```

### ON 800 HELD-OUT FRAMES

| | dark rule | network |
|---|---|---|
| right | 85.1 % | 95.1 % |
| obstacles missed | 19.7 % | 21.9 % |
| false stops | 13.5 % | 0.0 % |
| grey boxes missed | 70.5 % | 70.5 % |

*The dark rule calls anything darker than the floor in the lane blocked, so shadows stop the car. The network, trained on frames with both, does not stop for them; a grey box, as light as the floor, slips past both.*

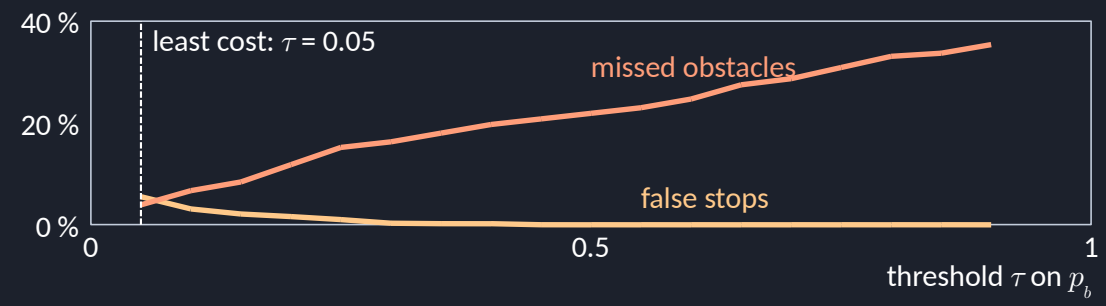
# The Cost of an Error

**Decision threshold** — the probability $\tau$ above which the car treats the path as blocked. A missed obstacle costs a collision; a false stop costs a second. The two errors are not equal, so $\tau$ is set by their costs, not at one half.

## THE EXPECTED COST OF A THRESHOLD

$$C(\tau) = c_{\text{miss}}\, P_{\text{miss}}(\tau) + c_{\text{stop}}\, P_{\text{stop}}(\tau)$$

$\tau$ the threshold on $p_b$, $P_{\text{miss}}, P_{\text{stop}}$ the shares of missed obstacles and false stops
$c_{\text{miss}} \gg c_{\text{stop}}$ : what each costs, a collision against a lost second




## CHOOSING THE THRESHOLD

| | |
|---|---|
| **a miss** | a collision: the run is lost |
| **a false stop** | a pause: a second is lost |
| **so** | set $\tau$ low, accept false stops |
| **the twin** | 21.9 % missed at 0.5, 3.9 % at 0.05 |
| **on the car** | measure both again |

*Lowering $\tau$ trades missed obstacles for false stops; the costs say where to stop.*

# Transfer Learning

**Transfer learning** — reusing a network trained on one task as the start of another. A ResNet-18 trained on ImageNet already detects edges, textures and shapes, so on the car only its head is new, and a few hundred frames of free and blocked are enough.

## THE ON-CAR RECIPE, IN FOUR LINES

BACKBONE KEPT, HEAD REPLACED

```
net = torchvision.models.resnet18(pretrained=True)   # the backbone, kept
net.fc = torch.nn.Linear(512, 2)                     # the new head, two outputs
x = normalize(frame, IMAGENET_MEAN, IMAGENET_STD)    # the backbone's statistics
opt = torch.optim.Adam(net.parameters(), lr=1e-4)    # fine-tune it all
```

Ex_11's small network trains from nothing, because its frames are drawn in thousands; on the car a few hundred real frames meet a reused backbone.

## THE SILENT ERROR

| | |
|---|---|
| **forget it** | and the model still trains |
| **the loss** | still falls |
| **the accuracy** | mediocre, for no visible reason |
| **so** | it is the first thing to check |

*The backbone expects its own mean and standard deviation, channel by channel; fed raw pixels it sees every frame through the wrong glass.*

# Deciding the Action

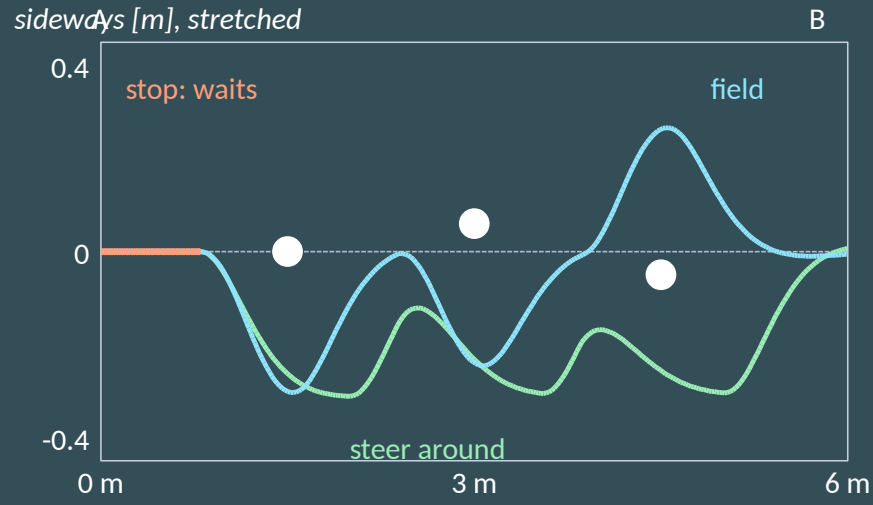
**Avoidance strategy** — the rule that turns what the car sees into what it does: stop, steer around, or follow a field around the obstacle. It is chosen first, because it decides what the network must tell: that, which side, or where.

## THE STRATEGIES, AND WHAT EACH NEEDS

| strategy | the car needs | reaches B? |
|---|---|---|
| stop | free or blocked | not past an obstacle that stays |
| steer around | blocked, and which side | yes, slowed down |
| field | where the obstacle is | yes, with a smooth path |

**TALK · 2 MIN** How would you decide, in your own words, between stopping and steering around? Then, in the paths on the right, why does stopping never reach B?

## THE THREE ON EX_11'S COURSE, 10 HZ




*Three obstacles on a 6 m course, each 16 cm across; the car is 20 cm wide. Course and sizes assumed.*

# Steering Around

**Avoidance rule** — the decision written as mathematics. The steer-around rule switches the lane aimed at when the network says blocked; the potential field adds a push away from every obstacle to the pull back to the line, and the car aims where the sum is lowest.

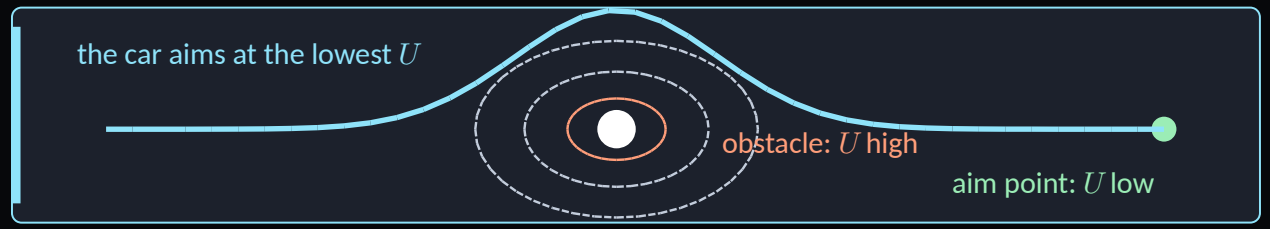
## THE RULE, AND THE FIELD

$$y_{\text{ref}} = \pm 0.30\, s, \qquad v = v_0\,(1 - 0.4\, s)$$

$s$ 1 when $p_b \geq \tau$, 0 when free, $v_0$ the cruising speed
$y_{\text{ref}}$ the side offset of the lane aimed at, away from the obstacle

$$U(y) = y^2 + \sum_i \frac{K}{d_i(y)}, \qquad y^* = \arg\min_y U(y)$$

$y$ the sideways position one aim distance ahead, $d_i(y)$ the gap to obstacle $i$
$K$ the strength of the push, $y^*$ the point the car aims at : the lowest $U$




## MEASURED ON THE COURSE

| | time to B | closest gap |
|---|---|---|
| stop, 10 Hz | never | 22.2 cm |
| steer, 5 Hz | 8.4 s | 2.3 cm |
| steer, 10 Hz | 8.3 s | 0.6 cm |
| field, 10 Hz | 6.3 s | 6.7 cm |
| field, 30 Hz | 6.2 s | 3.7 cm |

*A negative gap is a collision. The field is faster because it never slows down, but it needs where the obstacle is - a detector or a range sensor - not only free or blocked.*

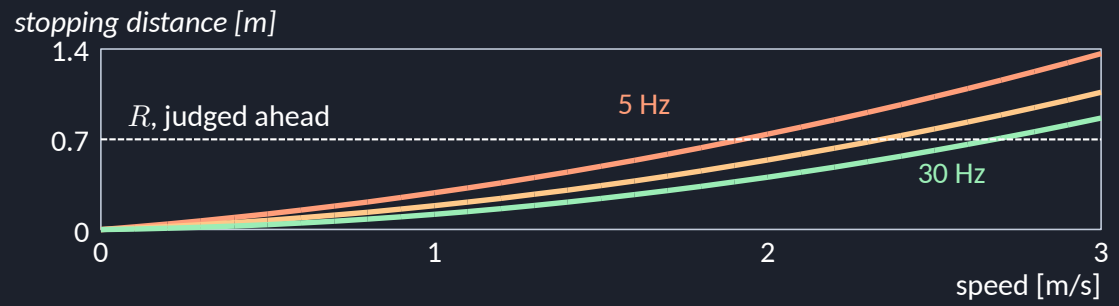
# Stopping Distance

**Stopping distance** — the distance the car covers from the frame that shows the obstacle to standing still: the blind distance of one decision, then the braking the tyres allow. It must be shorter than the range ahead that the camera judges.

## THE CONDITION FOR A SAFE SPEED

$$d_{\text{stop}} = \frac{v}{f} + \frac{v^2}{2\mu g} \leq R$$

$v$ the speed, $f$ decisions per second, $\mu$ the tyres' friction
$R$ how far ahead the camera judges free or blocked




## THE SAFE SPEED, BY LOOP RATE

| | |
|---|---|
| **5 Hz** | 1.93 m/s |
| **10 Hz** | 2.34 m/s |
| **30 Hz** | 2.68 m/s |
| **range $R$** | 0.70 m, assumed |
| **friction $\mu$** | 0.6, assumed |

*A faster loop buys speed only up to the braking limit; a longer range buys more.*

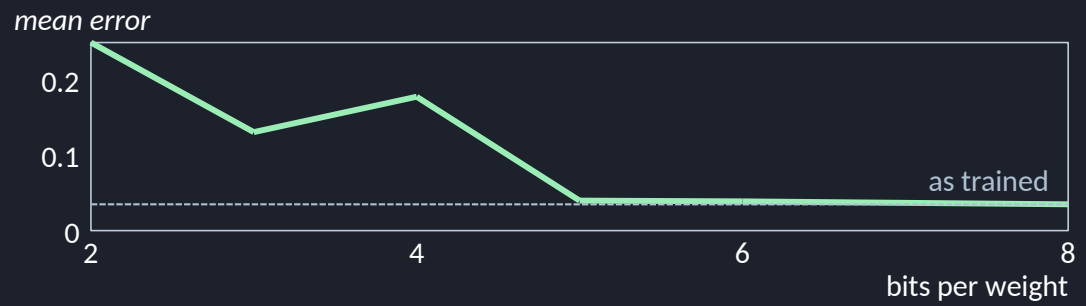
# Deployment on the Car

**Quantisation** — storing each weight in $b$ bits instead of 32: the range of the weights is cut into $2^b$ steps and every weight is rounded to its step. With compilation for the one GPU on the car, it is what makes the loop faster.

## THE ROUNDING, AND WHERE IT BREAKS

$$w_b = \Delta \,\mathrm{round}\left(\frac{w}{\Delta}\right), \qquad \Delta = \frac{w_{\max} - w_{\min}}{2^b - 1}$$

$w$ a weight, $w_b$ the weight stored in $b$ bits, $\Delta$ the step between the $2^b$ values kept




## COMPILED FOR THE DEVICE

| | |
|---|---|
| **train** | on a laptop or Colab |
| **move** | the weights only, not the model |
| **convert** | torch2trt, on the car itself |
| **half precision** | the large win on this GPU |
| **measure** | the whole loop, before and after |

*Five bits still steer; four do not. Find the cliff by measuring.*

# Advanced Topics

## RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic | what it adds | read | where it makes a difference |
|---|---|---|---|
| object detection | a box and a class for each object | UDL ch. 10 | crowded scenes, people |
| semantic segmentation | every free pixel of the floor | UDL ch. 10 | planning a path |
| path planning | a route on a map, not a reflex | UDL ch. 19 | warehouses, road cars |
| learned policies | the action learned from reward | UDL ch. 19 | simulators, legged robots |
| sensor fusion | a LiDAR or ultrasonic ranger too | UDL ch. 10 | the two mini projects |

### WHY THEY ARE LEFT OUT

Ex_11 classifies free or blocked from one camera on a known course, so none of them is needed to do it. The last row is the mini projects: the camera fused with a LiDAR, or with an ultrasonic ranger.

# Exercise 11

## ONE STUDY NOTEBOOK, FROM A TO B

- **1 – 4** — the frames, the detector, the steering network
- **5** — free or blocked: the dark rule and a network
- **6** — stop, steer around, field: the course from A to B
- **7** — the loop rate, the GPU and fewer bits
- **8 – 10** — the report, and the mini projects



*the same task on the real car: a demonstration in the lab*

## THE QUESTION EACH PART ANSWERS

| | |
|---|---|
| **1 – 4** | where is the path? |
| **5** | is the path blocked? |
| **6** | which strategy reaches B first? |
| **7** | how fast must the loop be? |

## REPORT QUESTION, TO CONCLUDE

Which avoidance strategy reaches B fastest without a collision, and what limits its speed: the errors of the networks, the loop rate, or the distance needed to react?

SUMMARY

# Key Takeaways

## OBSTACLE CLASSIFIER

| | |
|---|---|
| **what it is** | a CNN with two outputs, free and blocked |
| **function** | the probability that the path is blocked |
| **train, tune** | cross-entropy; the threshold by the costs |
| **vs a dark rule** | not fooled by shadows, misses fewer |
| **used for** | the first input of every strategy |

## AVOIDANCE POLICY

| | |
|---|---|
| **what it is** | the rule from what is seen to what is done |
| **function** | stop, steer around, follow a field |
| **train, tune** | chosen first: it sets what the network tells |
| **vs stopping** | reaches B; needs which side or where |
| **used for** | a safe speed: $v/f + v^2/(2\mu g) \leq R$ |

*Next — L12.1: the power grid, where a network estimates what the meters cannot see.*

# Questions

## 1 – 2 · IDENTIFYING AN OBSTACLE

**1** **How can a network decide from one camera frame whether the path is blocked, where a dark-pixel rule fails?**

The network has two outputs, free and blocked, turned into a probability by the softmax and trained with cross-entropy on frames labelled by where the obstacle stands: in the lane and near. A rule that calls anything darker than the floor blocked stops for shadows and misses grey obstacles. On the rendered frames the rule is right 85 % of the time and the network 95 %, with no stop for a shadow; both miss most obstacles as light as the floor. *UDL ch. 5; Ex_11 section 5*

**2** **Why is the threshold on the probability of a blocked path not set at one half, and how is it chosen?**

A missed obstacle costs a collision and a false stop costs a second, so the two errors have very different costs. Lowering the threshold trades missed obstacles for false stops, and the threshold is chosen where the expected cost, each error's rate times its cost, is smallest. On the rendered frames the network misses 22 % of the obstacles at one half and 4 % at 0.05, where it stops for 5.5 % of the free frames; on the car both rates are measured again. *UDL ch. 5; Ex_11 section 5*

## 3 – 4 · DECIDING THE ACTION

**3** **How does the avoidance strategy decide what the network must tell, and why does stopping alone not reach B?**

Stopping needs only free or blocked, but an obstacle that stays keeps the car standing, so it never reaches B. Steering around needs which side is free, and shifts the lane aimed at while the path is blocked, slowed to 60 %. A potential field adds a push away from every obstacle to the pull of the aim point, so it needs where each obstacle is, from a detector or a range sensor. On the course the field reaches B in 6.3 s, steering around in 8.3 s with half a centimetre to spare. *UDL ch. 19; Ex_11 section 6*

**4** **Why does the rate of the control loop limit the safe speed of a car that avoids obstacles?**

From the frame that shows the obstacle to standing still, the car drives one blind distance, its speed divided by the decisions per second, and then brakes as far as the friction of the tyres allows. That sum must stay within the range the camera judges. With a range of 0.7 m the safe speed is 1.9 m/s at 5 decisions a second and 2.7 m/s at 30; a faster network shortens only the first of the two distances. *Ex_11 section 7*

NEXT

# L12.1 — Power Grid Stability Estimation

The last block: the power grid, where a network proposes the state and the power flow equations judge it - with no true state to train on.

## BEFORE THE NEXT SESSION

| | |
|---|---|
| **read** | UDL ch. 5 |
| **run** | Ex_11's notebook, sections 5 to 7 |
| **bring** | your strategy's time to B, and its gap |
| **mini project** | the camera with a LiDAR, or ultrasonic |

*Block eleven is finished: a car that finds its path and a car that gets past what blocks it.*